ALGORITMA GENETIKA (GA)

Disusun untuk memenuhi Tugas Laporan Mata Kuliah Kecerdasan Buatan

Kelompok 14:

Izzan Maula Rifqi 103122430009

Muhammad Akbar Ivanka 103122400069

PROGRAM STUDI S1 SOFTWARE ENGINEERING

TELKOM UNIVERSITY PURWOKERTO

A. Deskripsi

Dalam Kecerdasan Buatan, algoritma pencarian stokastik heuristik seringkali diandalkan untuk memecahkan masalah optimasi yang kompleks. Permasalahan yang dikaji dalam studi kasus ini adalah pencarian nilai minimum dari sebuah fungsi matematis non-linear dua variabel. Objektif utama dari program ini adalah mengimplementasikan Algoritma Genetika (GA) tanpa bergantung pada library GA bawaan, guna menemukan nilai $x1$ dan $x2$ yang menghasilkan nilai evaluasi fungsi terkecil.
Fungsi matematis yang menjadi fungsi objektif didefinisikan sebagai berikut:

$$f(x1, x2) = -\left(sin(x1)cos(x2)\tan(x1 + x2) + \frac{1}{2}exp\left(1 - \sqrt{x}2^2\right)\right)$$

Adapun ruang pencarian dibatasi oleh domain (batas nilai) berikut:

- $-10 \leq x1 \leq 10$
- $-10 \leq x2 \leq 10$

B. Analisis dan Desain Algoritma Genetika

Untuk merancang algoritma yang dapat beroperasi secara optimal, penetapan arsitektur GA disusun melalui justifikasi desain berikut:

1. Rancangan Kromosom dan Metode Dekode

Representasi yang dipilih untuk memodelkan individu adalah pengkodean biner (binary encoding). Pengkodean biner digunakan karena dapat memberikan kontrol ketat terhadap presisi nilai desimal yang dihasilkan. Panjang kromosom ditetapkan sebesar 26 bit.
Kromosom tersebut dibelah menjadi dua segmen yang sama panjang:

- 13 bit pertama merepresentasikan gen untuk nilai $x1$.
- 13 bit kedua merepresentasikan gen untuk nilai $x2$.

Dengan 13 bit per variabel, terdapat atau 8.191 kemungkinan titik diskrit pada rentang -10 hingga 10 . Hal ini menghasilkan tingkat presisi koma desimal sebesar kurang lebih 0.0024 per titik, yang mana sudah sangat memadai untuk memetakan ruang pencarian secara akurat. Proses dekode dilakukan dengan mengonversi nilai biner masing-masing segmen menjadi desimal dasar, lalu memetakannya kembali ke batas domain aktual $[-10,10]$ menggunakan formulasi mapping linier baku.

2. Ukuran Populasi dan Kriteria Penghentian

Populasi ditetapkan berjumlah 125 individu per generasi. Karakteristik fungsi matematis yang memiliki elemen trigonometri (sin, cos, tan) cenderung menciptakan banyak puncak dan lembah minimum (local minima). Populasi sebesar 125 menjamin keanekaragaman genetik

(genetic diversity) yang luas di tahap awal pencarian, memperbesar probabilitas eksplorasi algoritma secara global.

Sementara itu, evolusi akan berjalan dan dibatasi hingga 50 generasi. Berdasarkan ukuran populasi yang cukup masif di awal, 50 iterasi evolusi diproyeksikan sudah mencukupi bagi kromosom untuk berkonvergensi secara stabil menuju titik minimum absolut tanpa mengorbankan terlalu banyak waktu komputasi.

3. Evaluasi dan Perhitungan Fitness

Algoritma Genetika secara natural berorientasi untuk memaksimalkan nilai fitness. Mengingat masalah ini adalah persoalan pencarian nilai minimum, maka nilai dari fungsi objektif perlu dimanipulasi. Rumus perhitungan fitness dirancang berbanding terbalik terhadap fungsi asli: Fitness = -$f(x1, x2)$

Dengan melakukan inversi nilai (mengalikannya dengan -1), nilai fungsi evaluasi yang semakin kecil (negatif) akan bertransformasi menjadi nilai fitness yang semakin besar (positif), sehingga algoritma dapat menyeleksinya sebagai kandidat terbaik.

4. Metode Pemilihan Orang Tua (Parent Selection)

Metode yang diterapkan untuk memilih individu sebagai parent adalah Seleksi kromosom dengan ukuran seleksi $k = 3$. Tiga individu diambil secara acak dari populasi, kemudian dibandingkan nilai fitness-nya. Individu dengan fitness tertinggi dari ketiga sampel tersebut akan menang dan terpilih sebagai orang tua. Metode ini dipilih karena mampu menjaga tekanan seleksi agar tetap seimbang, serta meminimalisir dominasi kromosom super di awal generasi yang sering menjadi kelemahan metode Roulette Wheel, sehingga membantu mencegah konvergensi dini.

5. Metode dan Probabilitas Operasi Genetik

Proses modifikasi genetik bergantung pada dua mekanisme utama:

- Crossover: Menggunakan metode Single-point Crossover dengan Probabilitas (Pc) sebesar 0.8 (80 %). Tingkat probabilitas yang tinggi (0.8) dirancang agar algoritma lebih proaktif mengeksplorasi kombinasi ruang pencarian baru dari individu-individu unggul. Metode pemotongan tunggal dipilih karena sederhana secara komputasi namun efisien untuk mencampur biner.
- Mutasi: Menggunakan metode Bit-flip Mutation dengan Probabilitas (Pm) sebesar 0.05 (5 %). Mutasi berfungsi sebagai injeksi keanekaragaman saat populasi mulai homogen dan terjebak di local minima. Nilai 0.05 dipilih agar pelestarian kromosom baik tidak rusak secara masif.

6. Metode Pergantian Generasi

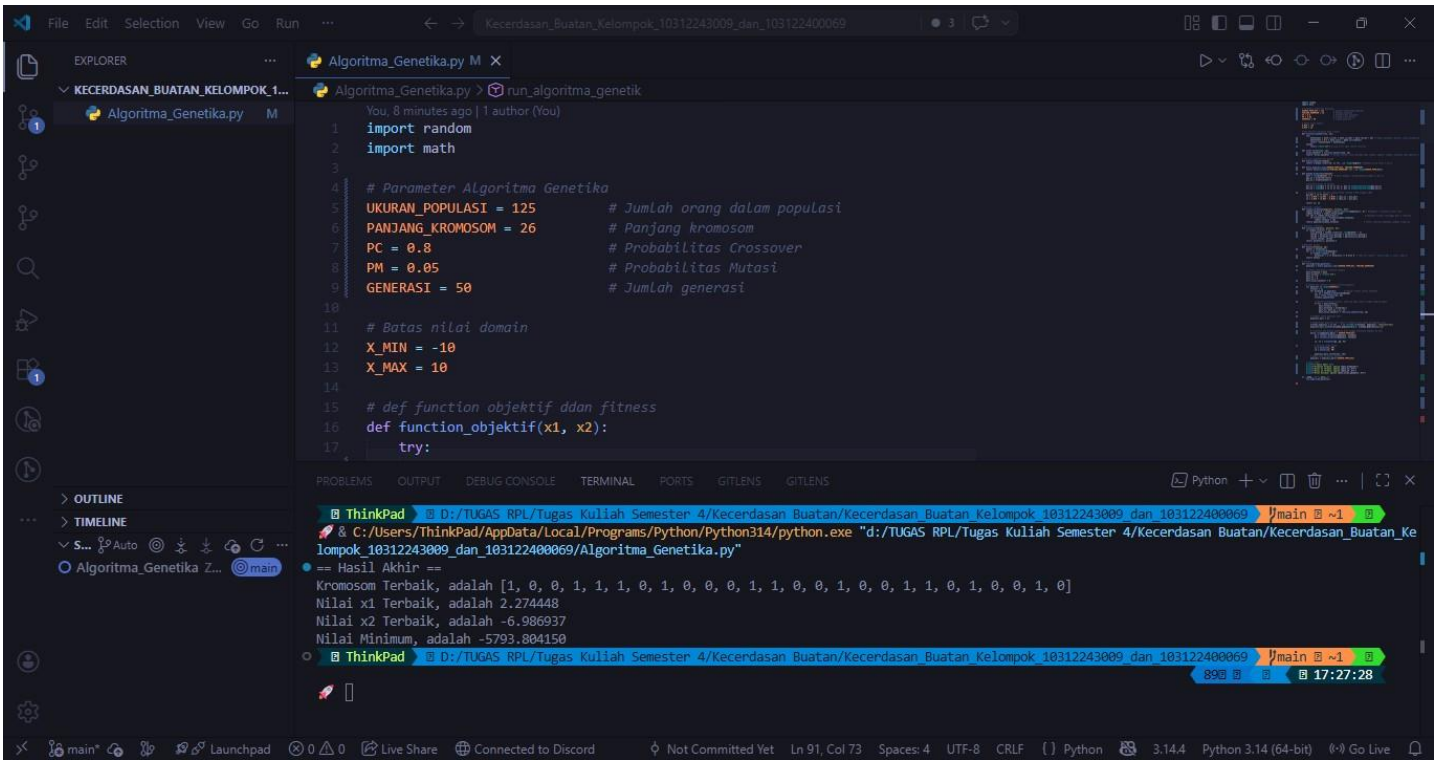
Pergantian generasi dikendalikan dengan strategi pergantian penuh (Generational Replacement) yang disempurnakan dengan metode Elitism. Setiap terjadi transisi ke generasi baru, seluruh populasi akan digantikan oleh keturunan baru hasil crossover dan mutasi, namun dua individu terbaik dari generasi saat itu akan langsung dikloning ke generasi berikutnya tanpa diubah sedikitpun.

Tujuan utama Elitism adalah sebagai jaring pengaman agar gen terbaik yang secara kebetulan telah ditemukan tidak hilang atau rusak akibat efek merusak yang mungkin terjadi pada operasi mutasi maupun crossover selanjutnya.

C. Implementasi Program

Kode program dibangun murni menggunakan Python dengan bantuan library standar yaitu math dan random untuk keperluan komputasi matematis dan probabilitas. Seluruh sub-sistem utama GA seperti inisialisasi populasi, komputasi fitness, seleksi kromosom, operasi mutasi, crossover, dan pembentukan generasi disusun secara algoritmik dari awal tanpa menggunakan Toolbox seperti DEAP atau PyGAD.

D. Output



E. Pembagian Peran Anggota Kelompok

Nama Anggota	NIM	Peran
Izzan Maula Rifqi	103122430009	Menganalisis parameter GA, menyusun arsitektur crossover dan mutasi, penulisan laporan.
Muhammad Akbar Ivanka	103122400069	Menulis implementasi Python, perbaikan bug, dan pengujian algoritma program.